

Agent Skills

实战使用指南

如何用 Skill 将 AI 编程助手变成专家团队

2026 年 5 月 | v1.0

一、认识 Skill

如果你用过的 AI 编程助手感觉“还行但不够好”，大概率是因为它没有加载 Skill。Skill 之于 AI，就像 App 之于手机——没有 App 的手机只能打电话发短信，没有 Skill 的 AI 只能做最基本的代码补全。

Agent Skill（智能体技能）是一种可安装的专业化能力模块。每个 Skill 封装了特定领域的知识、工作流程和工具脚本——PDF 处理、幻灯片制作、Bug 调试、代码审查……当你的任务匹配到某个 Skill 时，AI 会自动加载对应的“专家大脑”，以最佳方式完成任务。

Skill 系统的核心设计理念：按需加载，用完即走。不需要一次性把所有知识塞进提示词（那样既烧 Token 又降低质量），而是在正确的时刻激活正确的专家。

Skill 的四大价值

价值	说明
专业化	每个 Skill 在特定领域深耕，包含行业最佳实践
按需激活	只在匹配任务时加载，不占用不必要的上下文窗口
可共享	通过 GitHub 分发，团队共用一套 Skill 体系
跨平台	同一套 Skill 可用于 Claude Code、Codex、Copilot CLI 等

二、Skill 怎么工作

每个 Skill 在磁盘上就是一个 Markdown 文件（SKILL.md），结构极其简单。

文件结构

- 1. YAML Frontmatter（元数据）——声明 name（名称）和 description（触发条件描述）。description 是整个 Skill 最关键的部分，AI 通过它判断“这个 Skill 是否适用于当前任务”。
- 2. Markdown 正文——包含该领域的完整工作流程、工具说明、代码模板等。可以附带 scripts/（脚本）、templates/（模板）、references/（参考文档）等配套资源目录。

触发加载流程

整个过程对用户几乎透明，你只需要正常提问：

步骤	发生什么
1. 你提问	“帮我写一份周报 PPT”
2. 系统匹配	检测到 PPT 相关任务，匹配 pptx Skill
3. 加载 Skill	工具把完整 SKILL.md 内容注入 AI 上下文
4. AI 执行	AI 按照 Skill 指导的专业流程完成 PPT 制作

"即使只有 1% 的可能 Skill 适用，也必须先激活 Skill 再行动。这是不可协商的规则。"
—— Superpowers 核心原则

三、上手实战

来看三个真实场景，看看 Skill 在每一步是怎么干活的。

场景 1：写一份 PPT

你说：“帮我做一份 Q2 季度汇报 PPT。”

→ 系统激活 brainstorming Skill。先搞清楚 PPT 的受众是谁、要覆盖哪些数据、什么风格——不会一上来就写代码，而是先问清需求。

→ 设计确认后，激活 pptx Skill。按专业 PPT 制作流程生成 .pptx 文件，包含封面、目录、数据图表、总结页。

场景 2：改一个 Bug

你说：“这个登录功能报 500 错误，帮我看看。”

→ 系统激活 systematic-debugging Skill。不是瞎猜，而是走系统化调试流程——复现 Bug、看日志、定位根因、写测试复现、修复、验证。

→ 如果 Bug 涉及外部 API 调用，还可能激活 claude-api 或其他领域 Skill 辅助排查。

场景 3：审查代码

你说：“我刚写完这个模块，帮我看看有没有问题。”

→ 激活 requesting-code-review Skill。逐文件审查代码质量、安全性、性能、可维护性，给出结构化反馈。

→ 收到 Review 反馈后，激活 receiving-code-review Skill——不是每条意见都照改，先做技术验证，有道理的就改，存疑的就讨论。

关键点：每个场景都是“流程 Skill 先上，领域 Skill 后上”。先决定怎么做（brainstorming / debugging），再决定做什么（pptx / 具体代码）。

四、构建自己的 Skill

除了使用现成的 Skill，你还可以把自己团队的专属工作流封装成 Skill。整个过程由 skill-creator 指导。

六步创建法

阶段	做什么
1. 捕捉意图	明确 Skill 要解决什么问题，边界在哪
2. 编写草稿	用 Markdown 写 SKILL.md，Frontmatter 写好 name 和 description
3. 创建测试	写几个典型提示词，在启用 Skill 的情况下跑一遍
4. 评估结果	检查输出质量——Skill 是否被正确触发？文档是否符合预期？
5. 迭代优化	根据评估反复修改 SKILL.md，直到效果满意
6. 扩展测试	用更多样化的输入验证 Skill 的稳定性和触发准确率

SKILL.md 文件模板

最简单的 Skill 就长这样：

```
--- name: my-skill description: 1. 2. 3. ## - -
```

写好 description 的诀窍

description 决定了 Skill 什么时候被触发。写得宽会误触发，写得窄会不触发。好例子：“当用户要求创建、读取、编辑 Word 文档（.docx）时使用……”——有明确的触发词（.docx、Word 文档）和排除条件（非 PDF、非 Google Docs）。

五、常见问题 & 进阶技巧

Q: Skill 没有被触发？

检查三点：(1) 提问是否包含 Skill description 中的关键词；(2) description 是否写得不够精确导致匹配不上；(3) Skill 文件路径是否在系统扫描范围内。

Q: 多个 Skill 同时匹配怎么办？

流程型优先于实现型。brainstorming、debugging、writing-plans 这类“决定方法”的 Skill 总是最先加载。然后才是 pdf、pptx、frontend-design 这类“执行工作”的 Skill。如果同类型有多个候选，选最精准的那个。

Q: 流程型 vs 实现型怎么区分？

流程型：定义“怎么做事”（brainstorming、debugging、TDD）——先激活。
实现型：定义“怎么做出具体东西”（pdf、pptx、frontend-design）——后激活。
记一个口诀：“先问怎么干，再问干出啥”。

Q: 如何保持 Skill 最新？

Skill 持续演进。如果安装来源是 GitHub 仓库，定期 `git pull` 即可。也可以用 `skills-lock.json` 锁定版本，确保团队环境一致。

进阶：必知的 6 个核心 Skill

Skill	用途	何时必用
using-superpowers	Skill 系统入口	每次对话启动
brainstorming	需求探索与设计	任何创造性工作前
systematic-debugging	系统化调试	遇到任何 Bug
test-driven-development	测试驱动开发	实现功能前
writing-plans	任务规划	多步骤任务前
verification-before-completion	完成验证	声称“完成”前

Agent Skill 系统是将 AI 从“通用助手”升级为“专家团队”的关键基础设施。每个 Skill 都能为你省下数小时的试错时间。善用 Skill，让 AI 真正成为你的生产力倍增器。

